

Question 2

```
type help, copyright, credits or license for more information.
>>> from scapy.all import *
>>> p = Ether() / IP() / ICMP()
>>> p
<Ether type=IPv4 |<IP frag=0 proto=icmp |<ICMP |>>>
>>> █
```

On a affecté à la variable p l'encapsulation de notre paquet ip : ethertype protocol

Question 3

```
>>> p.summary
<bound method Packet.summary of <Ether type=IPv4 |<IP frag=0 proto=icmp |<I
CMP |>>>
>>> p.show
<bound method Packet.show of <Ether type=IPv4 |<IP frag=0 proto=icmp |<ICMP |>>>
>>> ls(p)
dst      : DestMACField          = 'ff:ff:ff:ff:ff:ff' ('None')
src      : SourceMACField        = '00:00:00:00:00:00' ('None')
type     : XShortEnumField       = 2048          ('36864')
--
version  : BitField (4 bits)     = 4              ('4')
ihl      : BitField (4 bits)     = None           ('None')
tos      : XByteField            = 0              ('0')
len      : ShortField            = None           ('None')
id       : ShortField            = 1              ('1')
flags    : FlagsField            = <Flag 0 (>)    ('<Flag 0 (>')
frag     : BitField (13 bits)    = 0              ('0')
ttl      : ByteField             = 64             ('64')
proto    : ByteEnumField         = 1              ('0')
chksum   : XShortField           = None           ('None')
src      : SourceIPField         = '127.0.0.1'    ('None')
dst      : DestIPField           = '127.0.0.1'    ('None')
options  : PacketListField       = []             ('[]')
--
type     : ByteEnumField         = 8              ('8')
code     : MultiEnumField (Depends on 8) = 0              ('0')
chksum   : XShortField           = None           ('None')
```

P.summary et P.show affiche l'encapsulation général de la trame alors que ls(p) affiche tous les détails (@MAC src et dst , @IP src et dst , longueur d'en tête etc)

Question 4

Valeurs par défaut :

@MAC dst : ff:ff:ff:ff:ff:ff

@MAC src: 00:00:00:00:00:00

@IP dst: 127.0.0.1

@IP src: 127.0.0.1

TTL : 64

ICMP : Type 8 (echo request)

Question 5

127.0.0.1 → 127.0.0.1 ICMP Echo (ping) request

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
FF	FF	FF	FF	FF	FF	00	00	00	00	00	00	08	00	45	00
00	1C	00	01	00	00	40	01	7C	DE	7F	00	00	01	7F	00
00	01	08	00	F7	FF	00	00	00	00						

3 protocols in packet:

Ethernet	IPv4	ICMP
----------	------	------

Question 8

```
>>> p[0].show
<bound method Packet.show of <Ether  dst=ff:ff:ff:ff:ff:ff src=00:1c:42:70:f5:a5 type=ARP
|<ARP  hwtype=0x1 ptype=IPv4 hwlen=6 plen=4 op=who-has hwsrc=00:1c:42:70:f5:a5 psrc=10.211
.55.4 hwdst=00:00:00:00:00:00 pdst=10.211.55.1 |>>>
>>>
```

Il s'agit d'une trame ethernet avec une requête ARP

Question 9

```
>>> p[0:3].show()
0000 Ether / ARP who has 10.211.55.1 says 10.211.55.4
0001 Ether / ARP is at 00:1c:42:00:00:18 says 10.211.55.1
0002 Ether / IP / UDP / DNS Qry "b'google.fr.'"
>>>
```

Question 10

```
>>> p.show()
###[ Ethernet ]###
dst      = 08:00:27:e7:98:2c
src      = 08:00:27:73:37:c6
type     = IPv4
###[ IP ]###
version  = 4
ihl      = None
tos      = 0x0
len      = None
id       = 1
flags    =
frag     = 0
ttl      = 64
proto    = icmp
chksum   = None
src      = 192.31.25.12
dst      = 192.31.25.1
\options \
###[ ICMP ]###
type     = echo-request
code     = 0
chksum   = None
id       = 0x0
seq      = 0x0
```

Script [Scanpings.py](#)

Requête DNS

```
(kali㉿kali)-[~]:lookup 172.18.26.101
$ nslookup rt-bethune.univ-artois.fr
Server: 172.18.26.101 valid syntax
Address: 172.18.26.101#53
File "stdin", line 1

Non-authoritative answer: rt-bethune.univ-artois.fr
Name: rt-bethune.univ-artois.fr
Address: 193.49.114.18 ok: invalid syntax
$ nslookup iut-rt.univ-artois.fr
File "stdin", line 1

(kali㉿kali)-[~]:lookup iut-rt.univ-artois.fr
$ nslookup iut-rt.univ-artois.fr
Server: 172.18.26.101 valid syntax
Address: 172.18.26.101#53

Non-authoritative answer:
iut-rt.univ-artois.fr canonical name = iutb-etu-gtr-25-9.univ-artois.fr.
Name: iutb-etu-gtr-25-9.univ-artois.fr
Address: 172.31.25.9
```

Avec Scapy

```
chksum = 0x7749 / ARP(dst=192.31.25.12)
src = 172.18.26.101
dst = 192.31.25.12
\options \
###[ UDP ]### dst = 192.31.25.12 / ARP(dst=192.31.25.12)
sport = domain
dport = domain
len = 67
$ chksum = 0xc322 / ARP(dst=192.31.25.12)
###[ DNS ]### expected
id = 0
qr = 1
$ p < opcode = QUERY / ARP(dst=192.31.25.12)
rsh: no aa expected = 0
tc = 0
$ p < rd = 1 / ARP(dst=192.31.25.12)
$ p < ra = 1 / ARP(dst=192.31.25.12)
rsh: no z expected = 0
ad = 0
$ p < cd = 0
$ p < rcode = ok: univ-artois.fr
server: qdcount = 1
address: ancount = 1
nscount = 0
non-auth: arcount = 0
name: \qd bethune \ univ-artois.fr
address: |###[ DNS Question Record ]###
| qname = 'rt-bethune.univ-artois.fr.'
| qtype = A
| qclass = IN
$ p < \an iut-rt \ univ-artois.fr
server: |###[ DNS Resource Record ]###
address: | rname = 'rt-bethune.univ-artois.fr.'
| type = A
non-auth: | rclass = IN
iut-rt: | ttl = 82633 canonical name = iutb-etu-gtr-25-9.univ-artois.fr.
name: | rdlength = None
address: | rdata = 193.49.114.18
ns = None
ar = None
```

Test des ports avec TCP

```
>>> rep[TCP].flags = 'SA'  
True  
>>> █
```

```
>>> p = IP(dst = '192.31.25.1')/TCP(dport = 20, flags = 'S')  
>>> rep[TCP].flags = 'SA'  
True  
>>> p = IP(dst = '192.31.25.1')/TCP(dport = 23, flags = 'S')  
>>> rep[TCP].flags = 'SA'  
True  
>>> p = IP(dst = '192.31.25.1')/TCP(dport = 80, flags = 'S')  
>>> rep[TCP].flags = 'SA'  
True  
>>> █
```